ProduCast: A Relational Model of Film Casting and Production Networks*

Karan Brahmbhatt
University of Victoria
Victoria, BC, Canada
karanbrahmbhatt@uvic.ca

Moksh Patel
University of Victoria
Victoria, BC, Canada
mokshpatel@uvic.ca

Abstract

ProduCast maps an ER model of film casting into six SQLite relations seeded with 12 people, 7 movies and 8 cast entries. Keys, `CHECK` constraints and cascading deletes enforce consistent links; a layered Flask app supports searching and editing without SQL. Code and reproducible setup: https://github.com/karanbe17/film-talent-kg.

1 Introduction

Flat film lists hide performance roles, repeated collaborations and deletion effects. We translate our ERD into SQLite [2] using standard SQL features [1], seed known films, and demonstrate CRUD operations through scripts and a Flask interface.

2 ER-to-Relational Mapping

Table 3 lists all six relations and constraints. Each uses a surrogate `Id` and named `UK_`/`FK_`/`CK_` constraints. `Person` and `Movie` represent entities; their required, unique names/titles are simplified natural keys that exclude namesakes and remakes. `Actor` and `Director` implement overlapping IS-A subclasses: each has a unique `PersonId` foreign key, and a person may belong to both. `MovieDirector` connects movies to directors; `PerformanceRole` connects movies to actors with `RoleName` and a `RoleType` restricted to Lead, Supporting or Cameo. `UNIQUE(MovieId, ActorId)` permits one role per actor per movie. Cascading foreign keys remove dependent subclass/link rows when their parent is deleted, not unrelated people or movies.

3 Architecture and Implementation

Figure 1 depicts four layers. Flask routes (`src/ui`) accept forms but contain no SQL. Services (`src/biz`) validate inputs and perform atomic multi-table operations, reporting violations as `BizError`. Parameterized queries and database initialization live in `src/dal`; every connection enables `PRAGMA foreign_keys = ON`. Paths derive from `__file__` in `src/cmn/config.py` rather than the working directory.

4 Populating and Querying the Database

Insertion. `02_seed.sql` inserts people, subclasses and movies before their links, resolving names and titles instead of fixed IDs. The application also reuses existing people through parameterized lookup:

```
SELECT Id FROM Person WHERE Name = ?;
INSERT INTO Person(Name) VALUES (?); -- if absent
```

Retrieval. Six queries in `03_queries.sql` are summarized in Table 2. This query counts distinct directors per actor:

*Code, SQL scripts and documentation: https://github.com/karanbe17/film-talent-kg

Table 1: Rows loaded by `python main.py`.

Relation	Rows	Note
`Person`	12	6 directors, 6 actors
`Actor`	6	subset of `Person` (IS-A)
`Director`	6	subset of `Person` (IS-A)
`Movie`	7	six genres
`MovieDirector`	7	one director per movie
`PerformanceRole`	8	7 Lead, 1 Supporting

Table 2: Selected `SELECT` queries and their results on the seeded database.

Question	SQL features	Result
Movies of one genre	`WHERE`, `LIKE`	Sci-Fi: *Inception*, *Star Wars*
Movies per director	`JOIN`, `GROUP BY`	Scorsese 2; five others 1
Cast size per movie	`JOIN`, `COUNT`	*The Godfather* 2; others 1
Actors with several directors	`HAVING`, `DISTINCT`	DiCaprio 2, De Niro 2
Genres shared with Scorsese	subquery `IN`	*Raging Bull*, *The Wolf of Wall Street*
Orphaned cast rows	`LEFT JOIN`, `IS NULL`	0 rows (cascade check)

```
SELECT P.Name, COUNT(DISTINCT MD.DirectorId) AS Directors
FROM Actor A JOIN Person P ON P.Id = A.PersonId
JOIN PerformanceRole R ON R.ActorId = A.Id
JOIN MovieDirector MD ON MD.MovieId = R.MovieId
GROUP BY A.Id, P.Name
HAVING COUNT(DISTINCT MD.DirectorId) > 1;
```

It returns DiCaprio (Nolan, Scorsese) and De Niro (Scorsese, Coppola); `DISTINCT` prevents duplicate director counts. **Modification.** Updates and deletes use `WHERE`; a representative test is:

```
UPDATE Movie SET Genre = 'Science Fiction' WHERE Title = 'Star Wars';
DELETE FROM Movie WHERE Title = 'Star Wars';
```

Deleting *Star Wars* changes Movie/MovieDirector/PerformanceRole counts from 7/7/8 to 6/6/7 without deleting people. Five tests reject a repeated person name, nonexistent director ID, invalid role type, repeated movie–actor pair and null title.

5 User Interface

Figure 2 shows four workflows. The Network Data Entry page (a) creates records through forms rather than requiring users to enter IDs. The Project Portfolio (b) lists movies and associated directors; Talent Rosters (c) expose actor and director records for editing or deletion. The Casting Ledger (d) displays movie–actor assignments with their character and role types. Input passes through the business layer, where services resolve entered names and titles to stored identifiers. Thus, the interface exposes the relationships in the ER model without presenting raw SQL to the user.

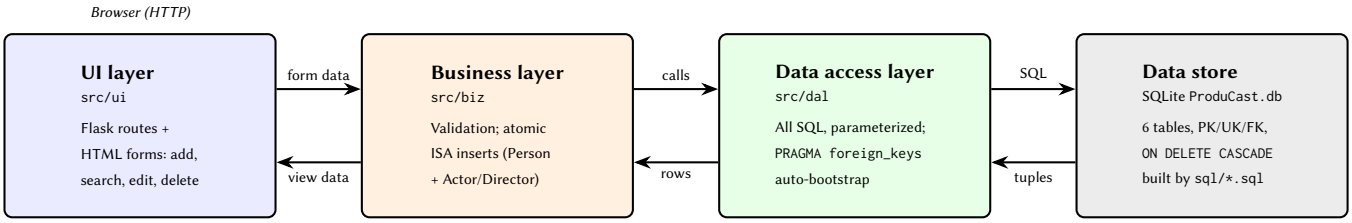
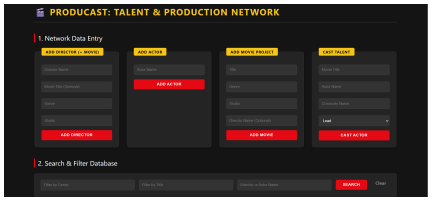
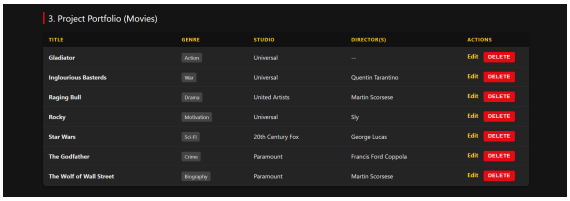


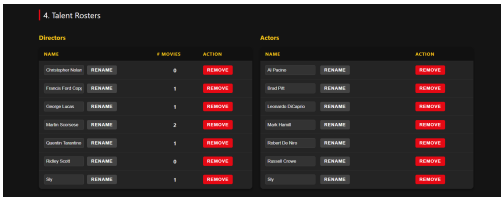
Figure 1: System architecture. Requests flow left to right (UI → business → data access → SQLite) and results flow back; only the data layer contains SQL.



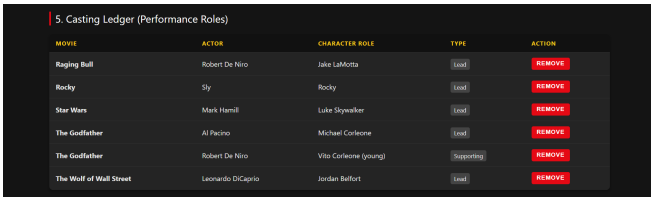
(a) Insertion: Network Data Entry forms (one per multi-table insert).



(b) Retrieval: Project Portfolio with its movies and directors.



(c) Management: Talent Rosters for the `Person` IS-A hierarchy (rename, delete).



(d) Result: Casting Ledger showing the `PerformanceRole` bridge.

Figure 2: The web application in use: insertion (a), portfolio retrieval (b), roster management (c) and ledger results (d).

6 Design Decisions

Surrogate `Id` keys simplify joins while unique names/titles prevent duplicates. Cascading deletes avoid manually removing orphaned links. `CHECK` restricts `RoleType`; service validation rejects empty strings that `NOT NULL` permits. Forms accept names and titles, which services resolve to IDs. This keeps displayed values readable while foreign keys retain stable links. The separation of routes, services and SQL also localizes data-integrity checks: a failed validation is reported before an invalid multi-table change is committed.

7 Validation of the Mapping

The sample uses six relations with 12 people, seven movies and eight cast links (Table 1). The director-count query traverses the performance and directing bridges; `DISTINCT` prevents duplicate counting and identifies DiCaprio and De Niro with two directors each. The orphan query returns no rows. Updating a movie's genre preserves its links, while deleting *Star Wars* removes its two dependent links but leaves people intact. Five rejection cases test duplicate names, missing directors, invalid roles, duplicate assignments and null titles. These checks demonstrate the principal constraints for the seeded instance, not exhaustive coverage.

8 Reproducibility and Discussion

The repository provides scripts, source, diagrams, screenshots and setup instructions, following FAIR [3]. Install with `pip install -r requirements.txt`; `python main.py` initializes and seeds the database, `–reset` rebuilds it and `–queries` runs examples. An earlier submission failed because a working-directory-dependent path created an empty SQLite database; corrected paths now pass tests from four directories. **Mapping limits.** Keys and participation constraints carry over, but relationship labels and overlapping IS-A meaning are not explicit in SQL. Unique names and one role per movie are assumptions; RDF/OWL in Part II will describe the semantics. **Contributions and AI.** K.B. developed the conceptual model/IS-A mapping; M.P. built the relational implementation. Gemini [5] generated an initial implementation that failed demonstration; Claude [4] refactored the layers and drafted scripts and report. We validated its output with tests because it could not access the repository.

References

[1] ISO/IEC. 2023. ISO/IEC 9075:2023 Information technology – Database languages SQL.
[2] SQLite Consortium. 2026. SQLite. https://www.sqlite.org.
[3] Mark D. Wilkinson et al. 2016. The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data* 3 (2016), 160018.
[4] Anthropic. 2026. Claude. AI help with refactoring, SQL scripts and report drafting.
[5] Google. 2026. Gemini. AI help with the first implementation.

Table 3: All relations, attributes and constraints (DDL in `src/dal/models/relational/sql/01_schema.sql`). PK = primary key (`INTEGER PRIMARY KEY AUTOINCREMENT`), UK = candidate key, FK = foreign key, CK = `CHECK`. ER origin gives the ERD construct each relation implements.

Relation (ER origin)	Attribute	Type	Null	Key	Constraint / meaning
`Person` (entity set; IS-A superclass)	`Id`	INTEGER	no	PK	surrogate key
	`Name`	VARCHAR(255)	no	UK	`UK_Person_Name UNIQUE (Name)`: natural key
`Actor` (subclass IS-A)	`Id`	INTEGER	no	PK	surrogate key
	`PersonId`	INTEGER	no	FK, UK	`FK_Actor_PersonId_2_Person_Id` (CASCADE); `UK_Actor_PersonId` (1:1 mapping)
`Director` (subclass IS-A)	`Id`	INTEGER	no	PK	surrogate key
	`PersonId`	INTEGER	no	FK, UK	`FK_Director_PersonId_2_Person_Id` (CASCADE); `UK_Director_PersonId` (1:1 mapping)
`Movie` (entity set)	`Id`	INTEGER	no	PK	surrogate key
	`Title`	VARCHAR(255)	no	UK	`UK_Movie_Title UNIQUE (Title)`: natural key
	`Genre`	VARCHAR(255)	no		mandatory text
	`StudioName`	VARCHAR(255)	yes		optional
`MovieDirector` (M:N bridge, Movie–Director)	`Id`	INTEGER	no	PK	surrogate key
	`MovieId`	INTEGER	no	FK	`FK_MovieDirector_MovieId_2_Movie_Id` (CASCADE)
	`DirectorId`	INTEGER	no	FK	`FK_MovieDirector_DirectorId_2_Director_Id` (CASCADE)
				UK	`UK_MovieDirector_MovieId_DirectorId UNIQUE (MovieId, DirectorId)`
`PerformanceRole` (M:N bridge, ERD `Performance_Role`)	`Id`	INTEGER	no	PK	surrogate key
	`MovieId`	INTEGER	no	FK	`FK_PerformanceRole_MovieId_2_Movie_Id` (CASCADE)
	`ActorId`	INTEGER	no	FK	`FK_PerformanceRole_ActorId_2_Actor_Id` (CASCADE)
	`RoleName`	VARCHAR(255)	no		the character played
	`RoleType`	VARCHAR(20)	no	CK	`DEFAULT 'Supporting'`; `CK_PerformanceRole_RoleType CHECK (RoleType IN ('Lead','Supporting','Cameo'))`
				UK	`UK_PerformanceRole_MovieId_ActorId UNIQUE (MovieId, ActorId)` (one role per actor and movie)

Referential integrity. Foreign-key enforcement is enabled on every connection (`PRAGMA foreign_keys = ON`) by `database.get_connection`. All foreign keys use `ON DELETE CASCADE`: deleting a `Person` removes its `Actor`/`Director` rows and their links; deleting a `Movie` removes only its `MovieDirector` and `PerformanceRole` rows. Movies and people are never deleted by cascade from a link.
Mapping assumptions. The specialization is overlapping (a person may have both an `Actor` and a `Director` row) and the application always creates the subclass row together with its `Person` row. Movies, actors and directors may exist before their links, but every link row must reference existing parents. A movie may have several directors or none; an actor has at most one role per movie.
Implementation checks. Each `Id` is `INTEGER PRIMARY KEY AUTOINCREMENT` and each foreign key is `INTEGER`. `NOT NULL` does not reject empty strings, so the business layer rejects blank values before they reach the database. The DDL and this report agree on keys, `NULL` values and cascading deletion.
Loaded instance. Person 12 · Actor 6 · Director 6 · Movie 7 · MovieDirector 7 · PerformanceRole 8.